

Class #4:



Outliers: Outliers are data points that are significantly different from the rest of the data.

In a Dataset:

Data: [10, 12, 11, 13, 1000]

Here, **1000** is an outlier because it is much larger than the other values.



Normalization

- Normalization is a technique used to scale or transform data into a specific range.
- It helps in making different features (variables) comparable and improves the performance of machine learning algorithms.



Types of Normalization:

1. **Min-Max Normalization:** Scales data to a fixed range, (usually 0 to 1 or -1 to 1)..

$$X_{\text{normalized}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Join @hmaracollege for more free courses 🙄

2. **Z-Score Standardization:** Transforms data to have a mean of 0 and a standard deviation of 1.

$$X_{\text{normalized}} = \frac{X - \mu}{\sigma}$$

(where μ is the mean and σ is the standard deviation)

Min-Max Normalization vs Z-Score Normalization (Standardization)

Criteria	Min-Max Normalization	Z-Score Normalization (Standardization)
Definition	Scales to a fixed range	Centers around mean, std = 1
Formula	$\frac{X - X_{\min}}{X_{\max} - X_{\min}}$	$\frac{X - \mu}{\sigma}$
Range	Fixed (e.g., [0, 1])	No fixed range
Outliers	Sensitive	Can handle outliers



Why do we need normalization for multiple features? :

- **Avoids Dominance:** Ensures no feature disproportionately influences the model.
- **Speeds up Convergence:** Helps gradient descent reach the minimum faster.

- **Improves Accuracy:** Makes distance-based models more reliable.
- **Prevents Numerical Instability:** Avoids calculation errors due to large values.

Normalization with single features :

- Helps with extreme values, speeding up learning.
- Makes hyperparameter tuning easier.

Effects with Activation Functions :

Activation Function	Without Normalization	With Normalization
Sigmoid	Saturates (0 or 1), causing vanishing gradients	Inputs stay in responsive range, better learning
Tanh	Saturates at -1 or 1, causing vanishing gradients	Responsive range maintained
ReLU	Exploding gradients with large positive values	Stable learning and consistent behavior
Softmax	Overconfident, near-binary outputs	Better-calibrated probability distributions

Join @hmaracollege for more free courses 🙄

Preprocessing

The process of preparing raw data for analysis by cleaning, transforming, and organizing it to improve model accuracy and efficiency.

Preprocessing Steps for Different Data Types

Data Type	Preprocessing Steps
Text Data	- Convert to lowercase: Standardize all text. - Tokenization: Break text into individual words/tokens. - Padding: Make all sequences the same length for batch processing. - Remove stopwords: Filter out common words like "the", "am", "to".
Image Data	- Scaling: Convert pixel values from 0-255 to a standardized range (e.g., 0-1). - Denoising: Remove unwanted noise. - Grayscale conversion: Reduce dimensions by converting to grayscale. - Data augmentation: Generate modified versions to expand the dataset.
Audio Data	- Silence removal: Remove silent portions. - Denoising: Filter out background noise. - Amplitude normalization: Standardize volume levels. - Frequency analysis: Extract relevant frequency features.
Tabular Data	- Handle null values: Impute missing data or remove rows. - Normalization: Scale numerical features to similar ranges. - Transformation: Apply log or other transformations for skewed data. - Remove irrelevant features, noise, duplicates: Clean the dataset. - Dimensionality reduction: Reduce the number of features while preserving information.

Tokenization, Encoding, and Embedding

 **Tokenization:** The process of breaking down text into smaller units called tokens, such as words or subwords.

Example: "Machine learning is fun" → ['Machine', 'learning', 'is', 'fun']

 **Encoding:** Converting tokens into numerical representations (integers) that models can process.

Example: ['Machine', 'learning', 'is', 'fun'] → [1, 2, 3, 4]

Join @hmaracollege for more free courses 🤖

Primary Types of Encoding		
Encoding Type	Description	Example
Label Encoding	Assigns a unique integer to each category.	Categories: ['Low', 'Medium', 'High'] Low → 0 Medium → 1 High → 2
One-Hot Encoding	Converts categories into binary vectors.	Categories: ['Red', 'Green', 'Blue'] Red → [1, 0, 0] Green → [0, 1, 0] Blue → [0, 0, 1]

 **Embeddings:** Embeddings are continuous vector representations of words or categories that capture their meanings and relationships in a lower-dimensional space.

How it works:

- Words or categories are represented as vectors in a multi-dimensional space
- Similar words have vectors that are closer together

Example: $\text{Vector}(\text{'king'}) - \text{Vector}(\text{'man'}) + \text{Vector}(\text{'woman'}) \approx \text{Vector}(\text{'queen'})$

Commonly used Embedding Models: Word2Vec, GloVe, BERT

Accuracy vs Loss

Metric	Definition	Measurement	Goal	Interpretation
Accuracy	Measures how often predictions are correct.	Percentage (0% to 100%)	Higher is better	Shows model's overall performance.
Loss	Measures the error in predictions.	Scalar value	Lower is better	Indicates model's error rate.

 **When we perform Loss Calculation then:**

- Binary Cross-Entropy works well with targets like 0 and 1, common in classification.
- MSE and MAE are sensitive to the scale of the target values. If targets have large ranges, normalization can improve performance and stability.

Convolution

Convolution is an operation that applies a filter (kernel) to an input (like an image) to extract features such as edges, textures, or patterns.

 **Modes of Convolution:**

- **Valid Convolution:**

- No padding is added, so the output is smaller than the input.
- The filter moves horizontally and vertically without going outside the input's borders.

Example: Input size 5x5 convolved with a 3x3 filter results in a 3x3 output.

- **Same Convolution:**

- Padding is added to keep the output size the same as the input.
- The filter slides horizontally and vertically, including padded borders.

Example: Input size 5x5 with a 3x3 filter and padding of 1 results in a 5x5 output.

- **Full Convolution:**

- Padding is added so that every element of the input is visited by the filter, resulting in a larger output.
- The filter starts outside the input boundary, moving horizontally and vertically over fully padded edges.

Example: Input size 5x5 with a 3x3 filter produces a 7x7 output.

Keras Overview

- **Keras:** High-level API for building neural networks, integrated into TensorFlow.

Common Keras APIs:

API	Description	Example
Sequential API	Linear stack of layers, simple and quick setup.	<code>model = tf.keras.Sequential([...])</code>
Functional API	More flexible, supports complex architectures.	<code>tf.keras.Model(inputs=..., outputs=...)</code>
Model Subclassing	Maximum flexibility using Python inheritance.	<code>class CustomModel(tf.keras.Model): ...</code>

Keras Components:

Join @hmaracollege for more free courses 🙏

Component	Description	Examples
Layers	Building blocks of models (Dense, Conv, LSTM).	<code>tf.keras.layers.Dense</code> , <code>tf.keras.layers.Conv2D</code>
Loss Functions	Measures model error (Cross-Entropy, MSE, MAE).	<code>loss='categorical_crossentropy'</code>
Optimizers	Controls learning (SGD, Adam, RMSprop).	<code>optimizer='adam'</code>
Callbacks	Functions triggered during training (e.g., EarlyStopping).	<code>tf.keras.callbacks.ModelCheckpoint</code>

A Simple model Lifecycle:

Stage	Description	Example
Model Creation	Define architecture, activation functions, input shapes.	<code>tf.keras.Sequential([...])</code>
Model Compilation	Specify optimizer, loss, and metrics.	<code>model.compile(optimizer='adam', loss='categorical_crossentropy')</code>
Model Training	Fit model to training data.	<code>model.fit(x_train, y_train, epochs=10)</code>
Model Evaluation	Evaluate performance on test data.	<code>model.evaluate(x_test, y_test)</code>
Model Prediction	Generate predictions on new data.	<code>model.predict(new_data)</code>
Model Saving/Loading	Save and load models for reuse.	<code>model.save('my_model.h5')</code>

Join @hmaracollege for more free courses 🙄